

ClusterFlowNet: 2D LiDAR Dynamic Object Segmentation and Velocity Estimation for F1TENTH Autonomous Racing

Jeongmin Im

Department of Robotics and Mechatronics Engineering
DGIST, Daegu, Korea
Email: jm154@dgist.ac.kr

Abstract—The perception of dynamic environments in high-speed autonomous racing necessitates algorithms that are both computationally efficient and robust to extreme sensor sparsity. While 3D LiDAR perception has achieved maturity, 2D planar LiDAR remains the sensor of choice for platforms like F1TENTH due to its high refresh rate and low weight. However, 2D data suffers from the “correspondence problem,” particularly during aggressive maneuvers where occlusion patterns shift rapidly, creating “newly appearing” points that lack temporal history.

This report presents a deep learning-based framework for *Cluster-based Velocity Estimation* on 2D LiDAR sequences. Unlike full-scan segmentation methods, we employ a lightweight PointNet-based architecture (ClusterFlowNet) that processes object clusters individually. We employ a targeted “New-Appearance Masking” (NAM) strategy that identifies points lacking temporal history (e.g., disocclusion) and masks them from the velocity regression loss to prevent gradient corruption. Furthermore, instead of relying on artificial loss weighting, we implement a rigorous dataset balancing strategy within our F1TENTH benchmark generation pipeline to address the inherent sparsity of dynamic agents. Experimental analysis demonstrates that our masking strategy, combined with a balanced training distribution, establishes a robust baseline for simultaneous segmentation and velocity estimation.

Index Terms—2D LiDAR, Velocity Estimation, F1TENTH, PointNet, Spatio-Temporal Masking, Autonomous Racing.

I. INTRODUCTION

2D planar LiDAR remains the primary sensor modality for a vast array of robotic applications, ranging from warehouse logistics (AGVs) to indoor service robots [1]. In these environments, the assumption of a planar world holds, making 2D sensors a cost-effective and computationally efficient alternative to their 3D counterparts. However, the safety of autonomous navigation in such shared spaces heavily relies on the ability to distinguish and predict the motion of dynamic agents among static infrastructure.

This necessity for dynamic perception becomes even more critical in the field of autonomous racing, which serves as a crucible for testing algorithms at the limits of handling. Platforms such as the F1TENTH vehicle require decision-making frequencies often exceeding 100 Hz [2]. In this domain, where latency is fatal, 2D LiDAR is dominant, prized for its ability to deliver precise range measurements at high frequencies (40–100 Hz) with minimal data overhead [3].

Despite these advantages, the reduction from 3D to 2D introduces profound ambiguity. Objects are collapsed into sparse line segments, and the lack of vertical texture makes correspondence matching notoriously ill-posed. This ambiguity is exacerbated during high-speed turns where the sensor’s Field of View (FoV) sweeps rapidly, introducing “new” points that did not exist in the previous frame.

For racing agents, accurately estimating the velocity of opponents is critical for overtaking maneuvers and collision avoidance [4]. However, the aforementioned data discontinuity poses a severe challenge. Traditional geometric consistency checks often fail because the ego-vehicle’s aggressive motion introduces massive global flow. Similarly, deep learning approaches struggle with the newly appearing points caused by rapid FoV changes, as standard architectures often fail to account for occlusion-induced correspondence breaks [6]. Forcing a network to regress velocity for a point with no history inevitably leads to “hallucinations” and noisy gradients.

This report proposes a Cluster-based Velocity Estimation Network (ClusterFlowNet) specifically tailored for 2D LiDAR. Our contributions are:

- 1) **New-Appearance Masking (NAM):** A practical loss-masking strategy that identifies points lacking temporal correspondence and explicitly excludes them from the loss function, stabilizing convergence.
- 2) **ClusterFlowNet Architecture:** A lightweight, PointNet-style architecture that inherits the core end-to-end velocity regression paradigm of FlowNet [5] and fuses local point features with global ego-motion context (v_x, v_y, ω) to robustly estimate relative velocity.
- 3) **Balanced Benchmark Dataset:** A curated dataset of 60k frames which not only addresses class imbalance but also includes geometric distractors to prevent shape-based overfitting.

II. RELATED WORKS

A. LiDAR-based Moving Object Segmentation (MOS)

Deep learning has revolutionized LiDAR perception. In the 3D domain, projection-based methods like RangeNet++ [7] and others project point clouds into spherical range images to leverage 2D CNNs. Recent works like HeLiMOS [8] utilize

residual images—differences between current and re-projected past scans—to detect motion. However, applying these 3D paradigms to 2D LiDAR is non-trivial. 2D scans lack the vertical dimension, reducing inputs to 1D sequences. While some approaches like TinyLidarNet [3] adapt 1D convolutions for binary classification (Static vs. Dynamic), they often neglect continuous velocity vector estimation, which is essential for predictive control in racing.

B. Scene Flow and Velocity Estimation

Velocity estimation is closely related to Scene Flow. Methods typically learn to regress flow vectors from sequential inputs. Pioneer deep learning architectures such as FlowNet [5] established the foundation for end-to-end optical flow estimation in the 2D image domain. However, directly applying such dense regression to sparse 2D LiDAR data remains challenging due to the *Aperture Problem*, where determining the tangential velocity of a featureless wall is mathematically impossible [4]. Existing networks often overfit to environmental context to resolve this. Our cluster-based approach mitigates this by focusing on object-level geometry rather than full-scene context.

C. Processing Unordered Point Sets

While CNNs require grid-structured data, LiDAR points are inherently sparse and unordered. PointNet-style architectures have established a standard for processing such data directly. In the context of sparse 2D LiDAR, datasets like Semantic2D [9] emphasize the importance of semantic understanding on sparse scans. Treating object segments as local point clouds allows for more efficient feature extraction compared to rasterizing sparse scans into high-resolution grids. Our ClusterFlowNet adopts this philosophy to maintain low latency for embedded hardware.

III. METHOD

A. Problem Formulation: Spatio-Temporal Correspondence

Velocity estimation relies on finding a corresponding point $p'_j \in S_{t-1}$ for a current point $p_i \in S_t$. Correspondence is broken by occlusion, FoV entry, or sparsity. **Definition 1:** A point p_i is “New” if no valid neighbor exists in the history S_{t-1} after ego-motion compensation.

$$L_{new}(p_i) = \begin{cases} 1 & \text{if } \nexists \text{ correspondence} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

The goal is to estimate the velocity vector $\mathbf{v}_i \in \mathbb{R}^2$ only when $L_{new}(p_i) = 0$.

B. ClusterFlowNet Architecture

Unlike full-scan CNNs that process the entire range image, our model operates on pre-clustered objects. The underlying paradigm of ClusterFlowNet is based on the core concept of FlowNet [5], adapted and scaled down from dense 2D image domains to sparse, object-level 2D LiDAR point sequences.

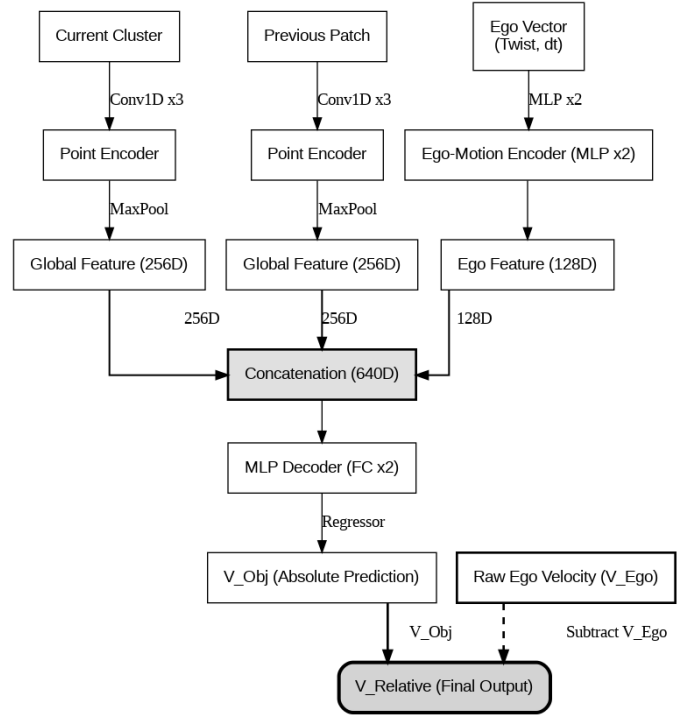


Fig. 1: Overview of the ClusterFlowNet architecture. The network takes a local point cluster ($N = 64$) and an ego-motion vector as input. A PointNet-based encoder extracts geometric features, which are concatenated with the ego-state context to regress the 2D relative velocity vector.

1) *Distance-based Clustering Strategy:* Prior to entering the network, raw LiDAR scans are grouped into object instances using Euclidean clustering (DBSCAN). We posit that for sparse 2D data, which lacks rich texture, spatial proximity is a sufficient proxy for instance segmentation. Although this geometric approach may over-segment continuous static environments (e.g., splitting a long wall into multiple clusters), it effectively isolates dynamic agents from the background. This trade-off is acceptable as our primary objective is the velocity estimation of discrete dynamic entities.

2) *Input Representation:* The network takes a fixed-size point set ($N = 64$) representing a single cluster. If a cluster has fewer points, we pad with zeros; if more, we downsample. The input tensor $X \in \mathbb{R}^{4 \times N}$ consists of:

- 1) **Local Coordinates** ((x, y)): Centered to the cluster mean and normalized to unit variance.
- 2) **Temporal Residual** (r_{res}): The difference between the measured range and the projected range from $t - k$, computed using odometry transformation. This is the primary cue for radial velocity.
- 3) **Normalized Angle** (θ_{norm}): Encodes the aspect angle of the cluster within the FoV to help resolving aspect-dependent distortions.

3) *Point Feature Encoder (Backbone):* We employ a PointNet-style encoder composed of shared MLPs (imple-

mented as 1D Convolutions). The layers expand the feature dimension progressively to capture geometric details:

- **Layer 1:** Conv1D(4 $\rightarrow$ 64) with Batch Normalization and ReLU.
- **Layer 2:** Conv1D(64 $\rightarrow$ 128).
- **Layer 3:** Conv1D(128 $\rightarrow$ 256).

A **Global Max Pooling** operation is then applied to aggregate per-point features into a single global cluster descriptor vector (1×256).

4) *Ego-Motion Context Injection:* Velocity estimation in racing is highly sensitive to the ego-vehicle’s own motion. To aid the network in disentangling ego-motion from object motion, we explicitly inject the ego-vehicle state ($v_x, v_y, \omega, \Delta t$) via a separate MLP branch (MLP 4 $\rightarrow$ 64 $\rightarrow$ 128). This context vector is concatenated with the cluster descriptor, resulting in a fused feature vector of size 384. This is passed through a decoding MLP (384 $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 2) to regress the final 2D relative velocity vector.

Augmentation for Robustness: To enhance robustness to real-world odometry drift and noise (a critical factor for Sim2Real transfer), we apply a novel data augmentation during training. The input linear velocity components (v_x, v_y) of the ego-motion vector are augmented with a small, state-dependent Gaussian noise ($\sigma \propto |v|$). This forces the network to learn velocity correction rather than assuming perfect odometry input.

C. New-Appearance Masking (NAM) Loss

To handle these temporal correspondence breaks systematically, we adopt a loss masking formulation that decouples valid estimation from unobservable data uncertainty.

1) *Hybrid Velocity Loss:* For a point i , the loss is computed only if it is not New ($y_{i,\text{new}} = 0$). We combine MSE with a Cosine Similarity term to enforce directional accuracy:

$$\mathcal{L}_{vel}^{(i)} = \|\mathbf{v}_i - \hat{\mathbf{v}}_i\|_2^2 + \lambda_{cos} \cdot (1 - \text{CosSim}(\mathbf{v}_i, \hat{\mathbf{v}}_i)) \quad (2)$$

where λ_{cos} balances the magnitude and direction errors.

2) *Standard Aggregation with Masking:* Unlike approaches that rely on artificial class weighting to handle imbalance, we rely on our balanced dataset strategy (described in Section III-D). Therefore, we employ a standard mean reduction over the valid points.

$$\mathcal{L}_{total} = \frac{\sum_{i=1}^N (1 - y_{i,\text{new}}) \cdot \mathcal{L}_{vel}^{(i)}}{\sum_{j=1}^N (1 - y_{j,\text{new}}) + \epsilon} \quad (3)$$

Here, the term $(1 - y_{i,\text{new}})$ acts as a hard gate. If a point is labeled as "New", the gradient for the velocity regression is strictly zeroed out. This prevents the network from forcing a prediction on unobservable data, while the denominator ensures the loss is normalized only by the number of valid points contributing to the gradient.

D. Dataset Construction and Balancing

We utilized the FITENTH simulator to collect high-fidelity physics data. To match the hardware constraint of the Hokuyo UTM-30LX LiDAR (25Hz), we applied a frame skipping strategy on 250Hz simulation data ($\Delta t = 40ms$).

1) *Addressing Class and Geometric Imbalance:* In a typical racing log, dynamic interactions are rare; over 90% of the data consists of static walls. Training on raw logs leads to a network that biases towards zero velocity. Furthermore, a naive network might learn a spurious correlation where "small object" equates to "dynamic." Consequently, instead of relying on artificial loss weighting, we opted for a **Dataset Balancing Strategy**:

- **Interaction Mining:** We filtered the raw simulation logs to identify frames containing dynamic objects within a 10m radius. Frames categorized as "Interaction" (Overtaking, Head-to-Head) were heavily oversampled relative to "Solo Driving" frames to ensure sufficient representation.
- **Geometric Regularization:** Crucially, to prevent overfitting to object shape, we intentionally included scenarios with static obstacles (e.g., pillars, cones) that share similar spatial dimensions with the racing agents. This forces the model to rely on temporal residuals rather than spatial appearance to infer velocity.
- **Resulting Distribution:** This curation resulted in a training set where approximately 40% of the clusters are dynamic, effectively mitigating mode collapse.

2) *Scenario Diversity:* We curated approximately 60,000 frames across three map topologies, explicitly modeling relative velocity scenarios:

- 1) **Map 1 (Aggressive Overtaking):** This map focuses entirely on overtaking scenarios. The ego-vehicle operates at high speeds, while the opponent vehicle is slower or blocks the path, necessitating aggressive high-speed passes. This creates rapid relative velocity changes.
- 2) **Map 2 (Low-Speed Receding):** The ego vehicle travels at a low constant speed (approx. 1 m/s), while the opponent accelerates away from the ego. This scenario tests the network’s ability to track objects moving away (receding) at extended ranges.
- 3) **Map 3 (Car-Following Separation):** This map includes interactions where the ego and opponent maintain equal speeds (car-following), testing stability in maintaining distance, or where the opponent moves away (receding).

IV. EXPERIMENTS AND RESULTS

A. Quantitative Analysis

We evaluated the performance of ClusterFlowNet using the validation split. Table I summarizes the velocity estimation error calculated from the validation set (excluding static frames where the car has not yet started moving).

TABLE I: Quantitative Analysis of Velocity Estimation Errors

Object Class	Mean Error (m/s)	Std Dev (m/s)
Static (Background)	0.22	0.41
Dynamic (Agents)	0.76	0.68

The low error for static objects (0.22 m/s) indicates that the model has successfully learned to compensate for the ego-

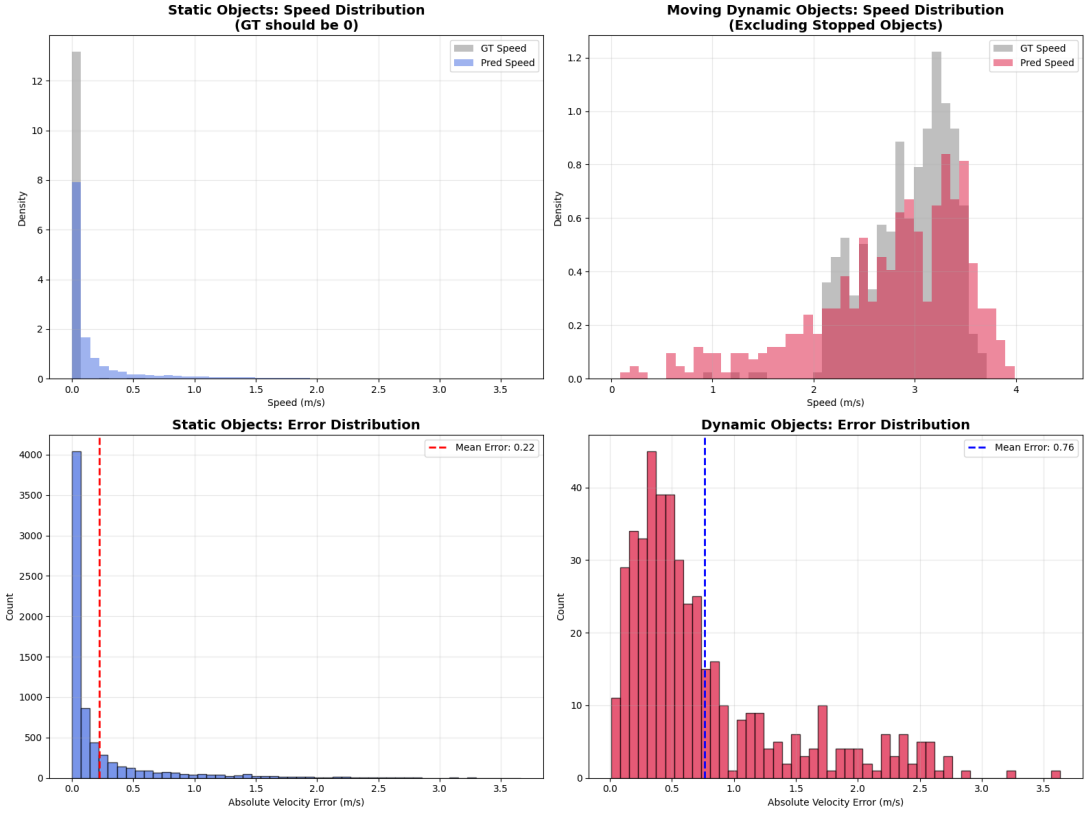


Fig. 2: Validation analysis of absolute velocity. (Top Row) Speed distributions for static objects (Left) and dynamic objects (Right). Static objects are tightly clustered near 0 m/s. (Bottom Row) Absolute Velocity Error (AVE) distributions. The graph shows a mean error of 0.22 m/s for static and 0.76 m/s for dynamic classes. Note the clear separation in predicted speed magnitudes between static and dynamic objects.

vehicle’s motion, effectively stabilizing the background. The dynamic object error is 0.76 m/s. This higher error is primarily attributed to the inherent sparsity of dynamic training samples. Despite our oversampling strategy, the absolute volume of dynamic data remains significantly lower than the dominant static background.

B. Qualitative Analysis

Fig. 2 presents the error distribution from the validation set. The analysis reveals that the static object error is concentrated near zero, and dynamic object predictions closely follow the ground truth trend, validating the effectiveness of the dataset balancing and NAM strategy. Specifically, in high-curvature turns where strictly geometric methods typically fail due to high rotational flow, our model maintains low error rates for static walls.

V. DISCUSSION

A. Generalization to Unseen Dynamics

To strictly evaluate the generalization capability of the network, we constructed a validation dataset with dynamics not seen during training. While the training set utilized discrete ego speeds (e.g., 3, 4, 5 m/s), the validation was performed on a held-out dataset where the ego-vehicle operates at an unseen

speed of 4.2 m/s. Furthermore, the opponent vehicle’s speed was modulated stochastically as $v_{opp} = 3.0 \text{ m/s} \times r$, where $r \sim \mathcal{U}(0.7, 1.2)$, creating a continuous range of opponent absolute velocities (approx. 2.1 to 3.6 m/s) that deviates from the training distribution. The robust results in Table I confirm that the explicit injection of the continuous ego-twist vector allows the model to interpolate effectively within the operational envelope.

B. Velocity Estimation as a Robust Segmentation Proxy

While precise velocity regression remains a challenging task, the qualitative results in Fig. 2 reveal a significant secondary benefit. There is a clear separation in the predicted speed distributions between static and dynamic classes. Predictions for static objects are consistently suppressed near zero (Mean Error 0.22 m/s), whereas predictions for dynamic objects predominantly populate the higher velocity spectrum ($> 1 \text{ m/s}$). This implies that even if the estimated velocity vector has some residual error (0.76 m/s), its magnitude serves as a highly robust feature for Dynamic Object Segmentation. Solving the harder problem of continuous regression implicitly solves the easier problem of binary classification (Moving vs. Static) with high reliability, potentially outperforming methods that only train on binary labels.

C. Verification of Masking Strategy

It is important to acknowledge that while the New-Appearance Masking (NAM) approach builds upon established principles of handling epistemic uncertainty, its specific contribution to the performance improvement has not been explicitly isolated in this study. We hypothesize that masking “New” points prevents the network from learning spurious correlations, but we have not yet experimentally compared the performance of ClusterFlowNet with and without the NAM component enabled via a dedicated ablation study. Consequently, the quantitative gains reported herein reflect the holistic performance of the proposed architecture and data generation strategy. Explicitly quantifying the marginal gain attributable solely to the masking logic remains a key objective for future work.

VI. CONCLUSION

This report presented ClusterFlowNet, a specialized architecture for 2D LiDAR velocity estimation in autonomous racing. By utilizing a cluster-based PointNet approach and introducing the New-Appearance Masking (NAM) strategy, we sought to address the fundamental correspondence problems inherent in sparse 2D data. Additionally, we demonstrated that a rigorous dataset balancing strategy—incorporating both interaction oversampling and geometric distractors—is crucial for preventing mode collapse and overfitting. The resulting benchmark and model establish a robust baseline for perception in high-speed racing scenarios, demonstrating that explicit geometric modeling and data balancing are sufficient to overcome the fundamental sparsity of 2D LiDAR without relying on expensive 3D sensors.

REFERENCES

- [1] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed., MIT Press, 2011.
- [2] M. O’Kelly, H. Zheng, D. Karthik, and R. Mangharam, “F1TENTH: An Open-source Evaluation Platform for Autonomous Racing,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2020.
- [3] H. Kim *et al.*, “TinyLidarNet: Lightweight 2D LiDAR-based End-to-End Deep Learning Model for F1TENTH Autonomous Racing,” *arXiv preprint arXiv:2410.07447*, 2024.
- [4] R. A. MacLachlan, “Tracking of Moving Objects with 2D LIDAR,” Carnegie Mellon University Robotics Institute Publications, 2005.
- [5] A. Dosovitskiy *et al.*, “FlowNet: Learning Optical Flow with Convolutional Networks,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2015.
- [6] B. Ouyang and D. Raviv, “Occlusion-Guided Scene Flow Estimation on 3D Point Clouds,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2021.
- [7] A. Milioto, I. Vizzo, J. Behley, and C. Stachniss, “RangeNet++: Fast and accurate lidar semantic segmentation,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2019.
- [8] H. Lim *et al.*, “HeLiMOS: A Dataset for Moving Object Segmentation in 3D LiDAR Data,” *arXiv preprint arXiv:2408.06328*, 2024.
- [9] Z. Xie and P. Dames, “Semantic2D: A Semantic Dataset for 2D Lidar Semantic Segmentation,” *arXiv preprint arXiv:2409.09899*, 2024.